Surypus ERP/CRM

A Structured Scientific Object for Reproducibility and Archivability

Domini

25 мая 2026 г.

Оглавление

Глава 1

Введение

This document serves as a structured scientific object containing the formally verified architecture, API semantics, and reproducible infrastructure of the Surypus ERP/CRM system. It is generated autonomously to ensure reproducibility and archivability.

Глава 2

System Architecture: Surypus ERP/CRM

2.1 Архитектурные принципы

Система построена на принципах разделения ответственности (Layered Architecture) и доменно-ориентированного проектирования (DDD).

2.1.1 Слои системы (Layer Separation)

1. **Domain**/: Чистые доменные модели, типы данных и инварианты. Не зависят от БД или API.
2. **Core/ (Service Layer)**: Бизнес-логика (расчет налогов, проведение документов, учет). Оркестрирует работу между DAL и внешними интерфейсами.
3. **DAL/ (Data Access Layer)**: Доступ к PostgreSQL через Hasql и Rel8. Содержит репозитории и Event Store.
4. **API/ (Handlers)**: REST эндпоинты (Scotty), преобразование JSON (Aeson) и Swagger спецификация.
5. **Infra**/: Утилиты, логирование, работа с JWT, WebSocket и интеграции.

2.2 Схема данных (Database Schema)

Основные домены в PostgreSQL:

- **RBAC**: `roles`, `permissions`, `user_roles`.
- **Inventory**: `goods`, `locations`, `stock`, `stock_movements`.
- **Accounting**: `accounts` (План счетов), `accounting_entries` (Проводки).
- **Documents**: `bills` (Документы), `bill_items` (Строки документов).
- **Service**: `jobs` (Очередь задач), `audit_log`, `schema_migrations`.

2.3 Event Sourcing

В Phase 3 внедряется Event Sourcing для критических изменений: - Все изменения состояния порождают события в таблице `event_store`. - Читаемые модели (projections) обновляются на основе потока событий. - Позволяет реализовать “Time Travel” аудит и надежную репликацию.

2.4 Безопасность (Security)

- **Аутентификация**: JWT (Access + Refresh tokens).
- **Авторизация**: RBAC middleware проверяет разрешения (`requirePermission`) перед выполнением handler-а.
- **Целостность**: Формальная верификация через LiquidHaskell для финансовых расчетов (внедряется).

2.5 Генерация отчетов

Используется гибридный подход: - **PDF-Slave**: YAML-шаблоны для быстрой генерации стандартных форм. - **JasperReports**: Сложные аналитические отчеты.

Последнее обновление: 2026-05-14

Глава 3

Surypus ERP API Documentation

3.1 Overview

The Surypus ERP system provides a RESTful API for managing accounting, inventory, tax, and reports. The API is built using Scotty and provides endpoints for core ERP functionality.

3.2 Base URL

`http://localhost:8080`

3.3 Authentication

The API uses JWT-based authentication. Include the JWT token in the `Authorization` header:

```
Authorization: Bearer <token>
```

3.4 Endpoints

3.4.1 Health Check

GET /health

Health check endpoint to verify the API is running.
Response:

```
OK
```

3.4.2 Accounting API

GET /api/v1/accounting/ledgers

Retrieve all ledgers.
Response:
[] { "status

: "operational
, "ledgers
: [] }

POST /api/v1/accounting/transactions

Create a new accounting transaction.
Response:
[] { "status
: "success
, "message
: "Transaction created (stub)
}

GET /api/v1/accounting/balance/:accountId

Get the balance for a specific account.
Parameters: - `accountId` (path parameter): Account ID
Response:
[] { "accountId
: "123
, "balance
: 0, "status
: "operational
}

3.4.3 Inventory API

GET /api/v1/inventory/goods

Retrieve all goods in inventory.
Response:
[] { "status
: "operational
, "goods
: [] }

POST /api/v1/inventory/goods

Create a new goods item.
Response:
[] { "status
: "success
, "message
: "Goods created (stub)
}

GET /api/v1/inventory/stock/:goodsId

Get stock level for a specific goods item.
Parameters: - `goodsId` (path parameter): Goods ID
Response:
[] { "goodsId
: "456
, "quantity
: 0, "status
: "operational
}

POST /api/v1/inventory/stock/movement

Record a stock movement (in/out).
Response:
[] { "status
: "success
, "message
: "Stock movement recorded (stub)
}

3.4.4 Tax API

GET /api/v1/tax/rates

Retrieve all tax rates.
Response:
[] { "status
: "operational
, "rates
: [] }

POST /api/v1/tax/calculate

Calculate tax for a given amount.
Response:
[] { "status
: "success
, "taxAmount
: 0, "message
: "Tax calculated (stub)
}

3.4.5 Reports API

GET /api/v1/reports/balance-sheet

Generate a balance sheet report.
Response:

[] { "status
: "operational
, "message
: "Balance sheet report (stub)
}

GET /api/v1/reports/income-statement

Generate an income statement report.

Response:

[] { "status
: "operational
, "message
: "Income statement report (stub)
}

GET /api/v1/reports/inventory

Generate an inventory report.

Response:

[] { "status
: "operational
, "message
: "Inventory report (stub)
}

3.4.6 Integration API

POST /api/v1/integrations/bank-statement/upload

Upload a bank statement (OFX or ISO20022 format).

Request Body:

[] { "content
: "bank statement content
, "format
: "OFX
}

Response:

[] { "importId
: "import-tenant-id
, "rowCount
: 0, "status
: "success
, "transactions
: [] }

GET /api/v1/integrations/health

Get health status of the integration system.

Response:

[] { "status
: "healthy
, "tenantId
: "default-tenant
, "healthData
: {} }

GET /api/v1/integrations/status

Get the operational status of integration endpoints.

Response:

[] { "status
: "operational
, "tenantId
: "default-tenant
, "endpoints
: [{ "path
: "/api/v1/integrations/bank-statement/upload
, "status
: "available
}, { "path
: "/api/v1/integrations/health
, "status
: "available
}, { "path
: "/api/v1/integrations/status
, "status
: "available
}] }

3.5 Running the Server

To start the API server:

[] stack run

The server will start on port 8080 by default.

3.6 Testing

Run the test suite:

[] stack test

3.7 Status

Current implementation status: - Accounting API (stub endpoints) - Inventory API (stub endpoints) - Tax API (stub endpoints) - Reports API (stub endpoints) - Integration API (functional) - Scotty Web Server

Note: Most endpoints are currently stub implementations and return placeholder data. Full implementation is planned for future phases.

Глава 4

Role-Based Access Control (RBAC) System

4.1 Overview

Surypus implements a comprehensive Role-Based Access Control (RBAC) system to manage user permissions across the API. This document outlines the role hierarchy, permissions, and how they are applied to API endpoints.

4.2 Role Hierarchy

The system defines four primary roles with different permission levels:

4.2.1 1. Admin (RoleAdmin)

- **Description**: Full system access
- **Permissions**: All permissions
- **Typical Users**: System administrators, developers
- **Key Permissions**: AdminAccess, all CRUD operations

4.2.2 2. Manager (RoleManager)

- **Description**: Business operations management
- **Permissions**: Most write operations, all read operations
- **Typical Users**: Department managers, supervisors
- **Limited By**: Cannot access admin functions, cannot delete sensitive data

4.2.3 3. User (RoleUser)

- **Description**: Standard user with basic read and write access
- **Permissions**: Limited write operations, basic read operations
- **Typical Users**: Regular employees, data entry staff
- **Limited By**: Read-only for sensitive areas (accounting, payroll), no delete permissions

4.2.4 4. Viewer (RoleViewer)

- **Description**: Read-only access for reporting and analysis
- **Permissions**: Read-only across the system
- **Typical Users**: External stakeholders, auditors, analysts
- **Limited By**: Cannot perform any write or delete operations

4.3 Permission Types

Permissions are organized by resource type:

4.3.1 Person Management

- `PersonRead` - Read person data
- `PersonWrite` - Create/update person data
- `PersonDelete` - Delete person records

4.3.2 Goods & Inventory

- `GoodsRead` - Read goods catalog
- `GoodsWrite` - Create/update goods
- `GoodsDelete` - Delete goods
- `StockRead` - Read stock information
- `StockWrite` - Modify stock (internal use)

4.3.3 Billing

- `BillRead` - Read bills/invoices
- `BillWrite` - Create/update bills
- `BillDelete` - Delete bills
- `BillPost` - Post/confirm bills

4.3.4 Locations & Warehouse

- `LocationRead` - Read location data
- `LocationWrite` - Create/update locations
- `LocationDelete` - Delete locations

4.3.5 Accounting & Finance

- `AccountingRead` - Read accounting entries
- `AccountingWrite` - Create/update accounting entries
- `PaymentRead` - Read payment information
- `PaymentWrite` - Create/update payments
- `PaymentDelete` - Delete payments

4.3.6 Orders & Operations

- `OrdersWrite` - Manage orders (read/write/delete)

4.3.7 Taxes & Currencies

- `TaxesWrite` - Manage tax rates and calculations
- `CurrenciesWrite` - Manage currency settings

4.3.8 Users & Settings

- `UsersRead` - Read user information
- `UsersWrite` - Create/manage users
- `SettingsRead` - Read system settings
- `SettingsWrite` - Modify system settings

4.3.9 Payroll

- `PayrollRead` - Read payroll data
- `PayrollWrite` - Create/update payroll
- `SalariesWrite` - Manage salary information

4.3.10 Reports & Analysis

- `ReportsRead` - Access reports
- `ReportsWrite` - Create/manage reports

4.3.11 Administrative

- `AdminAccess` - Access administrative functions

4.4 API Endpoint Permissions

4.4.1 Authentication Endpoints (No Auth Required)

```
POST /v1/login          - Public (no auth required)
POST /v1/refresh        - Public (no auth required)
```

4.4.2 Protected Endpoints

Persons

```
GET /v1/persons                - PersonRead
POST /v1/persons               - PersonWrite
GET /v1/persons/:id            - PersonRead
PUT /v1/persons/:id            - PersonWrite
DELETE /v1/persons/:id         - PersonDelete
GET /v1/persons/search/:query  - PersonRead
```

Goods

```
GET /v1/goods           - GoodsRead
POST /v1/goods          - GoodsWrite
GET /v1/goods/:id       - GoodsRead
PUT /v1/goods/:id       - GoodsWrite
DELETE /v1/goods/:id    - GoodsDelete
```

Locations

```
GET /v1/locations               - LocationRead
POST /v1/locations              - LocationWrite
GET /v1/locations/:id           - LocationRead
PUT /v1/locations/:id           - LocationWrite
DELETE /v1/locations/:id        - LocationDelete
```

Bills

```
GET /v1/bills                   - BillRead
POST /v1/bills                  - BillWrite
GET /v1/bills/:id               - BillRead
PUT /v1/bills/:id               - BillWrite
DELETE /v1/bills/:id            - BillDelete
PUT /v1/bills/:id/status        - BillPost
```

Payments

```
GET /v1/payments                - PaymentRead
POST /v1/payments               - PaymentWrite
GET /v1/payments/:id            - PaymentRead
PUT /v1/payments/:id            - PaymentWrite
DELETE /v1/payments/:id         - PaymentDelete
```

Orders

```
GET /v1/orders                  - OrdersWrite (for access)
POST /v1/orders                 - OrdersWrite
GET /v1/orders/:id              - OrdersWrite (for access)
PUT /v1/orders/:id/status       - OrdersWrite
DELETE /v1/orders/:id           - OrdersWrite
```

Taxes

```
GET /v1/taxes                   - TaxesWrite (for access)
POST /v1/taxes                  - TaxesWrite
GET /v1/taxes/:id               - TaxesWrite (for access)
PUT /v1/taxes/:id               - TaxesWrite
DELETE /v1/taxes/:id            - TaxesWrite
```

Currencies

```
GET /v1/currencies              - CurrenciesWrite (for access)
POST /v1/currencies             - CurrenciesWrite
GET /v1/currencies/:id          - CurrenciesWrite (for access)
PUT /v1/currencies/:id          - CurrenciesWrite
DELETE /v1/currencies/:id       - CurrenciesWrite
```

Stock & Inventory

```
GET /v1/stock                   - StockRead
GET /v1/stock/summary           - StockRead
GET /v1/stock/:goodsId/:locId   - StockRead
GET /v1/stock/goods/:goodsId    - StockRead
```

Accounting

```
GET /v1/accounting/accounts         - AccountingRead
POST /v1/accounting/accounts        - AccountingWrite
GET /v1/accounting/accounts/:id     - AccountingRead
PUT /v1/accounting/accounts/:id     - AccountingWrite
DELETE /v1/accounting/accounts/:id  - AccountingWrite

GET /v1/accounting/entries          - AccountingRead
POST /v1/accounting/entries         - AccountingWrite
GET /v1/accounting/entries/:id      - AccountingRead
PUT /v1/accounting/entries/:id      - AccountingWrite
DELETE /v1/accounting/entries/:id   - AccountingWrite
```

Payroll

```
GET /v1/payroll                 - PayrollRead
GET /v1/payroll/employees       - PayrollRead
GET /v1/payroll/employees/:id   - PayrollRead
GET /v1/payroll/salaries        - PayrollRead
GET /v1/payroll/salaries/:id    - SalariesWrite (for access)
```

Users

```
GET /v1/users           - UsersRead
POST /v1/users          - UsersWrite
```

RBAC Administration

```
GET /v1/rbac/roles              - AdminAccess
POST /v1/rbac/roles             - AdminAccess
PUT /v1/rbac/roles/:name        - AdminAccess
DELETE /v1/rbac/roles/:name     - AdminAccess
GET /v1/rbac/grants             - AdminAccess
POST /v1/rbac/grants            - AdminAccess
```

```
GET /v1/rbac/grants/active?principal=...              - AdminAccess
POST /v1/rbac/grants/cleanup                         - AdminAccess
PUT /v1/rbac/grants/:from/:to/:permission?resource=... - AdminAccess
DELETE /v1/rbac/grants/:from/:to/:permission?resource=... - AdminAccess
GET /v1/rbac/audit?principal=...&resource=...&offset=...&limit=... - AdminAccess
POST /v1/rbac/audit/cleanup?keep=...                  - AdminAccess
```

Dynamic roles are stored as scoped permissions and can be restricted to a canonical resource key such as `location:1`, `bill:42`, `report:sales`, or `accounting-entry:17`.

Delegation grants support temporary escalation through an expiry timestamp. Expired grants can be cleaned up explicitly through the cleanup endpoint. Updating a grant refreshes its expiry window while keeping its principal and scoped permission key. Audit entries can be paginated with `offset`/`limit` and trimmed with the audit cleanup endpoint.

Reports

```
GET /v1/reports                - ReportsRead
GET /v1/reports/metadata       - ReportsRead
GET /v1/reports/templates      - ReportsRead
GET /v1/reports/:id            - ReportsRead
GET /v1/reports/:name/jrxml    - ReportsRead
```

4.5 Implementation Details

4.5.1 Role-Permission Mapping

The system uses a permission matrix where each role is associated with specific permissions:

[] data RolePermission = RolePermission { rpRole :: Role , rpPermissions :: [Permission] }

4.5.2 Permission Checking

Permissions are checked using the `checkPermission` function:

[] checkPermission :: Text -> Permission -> Either String ()

This function: 1. Takes a role (as Text) 2. Takes a required permission 3. Returns `Right ()` if the role has the permission 4. Returns `Left error` if the role lacks the permission

4.5.3 API Integration

The `requirePermission` helper in `Surypus.API.Server` provides access checking:

[] requirePermission :: Text -> Permission -> Handler ()

This function returns a 403 Forbidden response if the permission check fails.

The runtime request pipeline now also supports a request-aware authorization resolver via advanced middleware. It can:

1. Authenticate non-public requests
2. Resolve required permission from the incoming request path/method
3. Load dynamic roles and delegation grants from an RBAC store
4. Apply resource-level checks using scoped permissions
5. Emit audit entries for allow/deny decisions

4.6 Usage Examples

4.6.1 Checking Admin Access

[] case checkPermission "admin
AdminAccess of Right () -> -- User is admin Left err -> -- User is not admin

4.6.2 Assigning Permissions to Endpoints

Endpoints use comments to document required permissions:
[] personsList :: Handler PersonsResponse -- | GET /v1/persons - Requires PersonRead permission personsList = return $PersonsResponse[...]1$

4.6.3 Using Roles in the Application

[] let userRole = roleFromText "manager
if hasPermission userRole PersonWrite then -- Allow write operation else -- Deny write operation

4.7 Testing

The RBAC system includes comprehensive tests in `test/RBACSpec.hs` and `test/Test.hs`:

- Role permission verification
- Cross-role permission boundaries
- Text-to-role conversion
- Permission checking functions

Run tests with:
[] stack test

4.8 Future Enhancements

1. **Dynamic Roles**: Create custom roles with specific permission sets
2. **Permission Delegation**: Allow managers to delegate specific permissions
3. **Audit Logging**: Track permission checks and changes
4. **Time-based Permissions**: Temporary permission escalation
5. **Resource-level Permissions**: Fine-grained access control per resource

4.9 Security Considerations

1. **Default Deny**: All permissions must be explicitly granted
2. **Role Validation**: Invalid roles default to the most restrictive (Viewer)
3. **Token-based Verification**: Permissions are verified from JWT tokens
4. **Permission Immutability**: Permissions are defined at compile time

4.10 Configuration

Role definitions are in `src/Surypus/RBAC.hs`. To modify permissions:

1. Update the `Permission` datatype
2. Add the permission to relevant `RolePermission` entries
3. Update endpoint documentation
4. Add corresponding tests

4.11 Support

For questions or issues with the RBAC system, please: 1. Check the test suite in `test/RBACSpec.hs` 2. Review endpoint documentation comments in `src/Surypus/API/Server.hs` 3. Consult the RBAC module documentation in `src/Surypus/RBAC.hs`

Глава 5

Development Guide: Surypus ERP/CRM

5.1 Подготовка окружения

5.1.1 Требования

- **GHC 9.8.4** (рекомендуется через `stack`)
- **Stack**
- **PostgreSQL 16+**
- **Docker & docker-compose** (для локальной БД)

5.2 Сборка и запуск

[] # Сборка проекта stack build

Запуск в режиме разработки (watch mode) stack build --file-watch

Запуск REPL (интерактивная среда) stack repl

5.3 Тестирование

Проект использует **Hspec** для Unit-тестов и **QuickCheck** для Property-based тестов.

[] # Запуск всех тестов stack test

Запуск тестов, соответствующих шаблону stack test --test-arguments -match 'VAT'

Запуск с подробным выводом stack test --verbose

5.3.1 Тестовая база данных

Для интеграционных тестов требуется запущенный PostgreSQL. См. `OPERATIONS.md` для настройки.

5.4 Стандарты кода

5.4.1 Стиль (Style Guide)

- **Индентация**: 2 пробела (табы запрещены).
- **Длина строки**: макс. 100 символов.
- **Предупреждения**: Весь код должен компилироваться с `-Wall` без предупреждений.

5.4.2 Именование

- **Модули**: PascalCase (`Core.Tax`)
- **Типы/Конструкторы**: PascalCase (`TaxRate`, `Asset`)
- **Функции/Поля**: camelCase (`calcVAT`, `trRate`)

5.4.3 Работа с типами

- Используйте `Text` вместо `String`.
- Используйте `Int64` для ID из базы данных.
- Используйте `Newtype` для оберток доменных понятий (`Money`, `BillID`).

5.5 Завершение сессии (Session Completion)

ВАЖНО: Работа считается завершенной только после успешного выполнения `git push`.

5.5.1 Обязательный процесс:

1. **Зафиксировать остаток работ**: Создать задачи в Beads для всего, что требует продолжения.
2. **Проверка качества**: Запустить тесты и линтеры.
3. **Обновить статус задач**: Закрыть выполненные задачи в Beads.
4. **PUSH В РЕПОЗИТОРИЙ**: `bash git pull --rebase git push`
5. **Верификация**: Убедиться, что `git status` показывает “up to date with origin”.

Последнее обновление: 2026-05-14

Глава 6

Operations Guide: Surypus ERP/CRM

6.1 Docker

Для локальной разработки и развертывания используются Docker и Docker Compose.
[] # Запуск всей инфраструктуры (DB, API, Worker) docker-compose up -d
Просмотр логов docker-compose logs -f api

6.1.1 Сервисы

- `db`: PostgreSQL 16 (порт 5432)
- `api`: Haskell REST Server (порт 8080)
- `worker`: Фоновый обработчик задач (отчеты, экспорт)

6.2 Миграции базы данных

Система использует кастомный механизм миграций.
Путь к файлам: `sql/migrations/`
[] # Инициализация и запуск всех миграций ./sql/migrations/init_db.sh

6.2.1 Порядок добавления новой миграции

1. Создайте файл `sql/migrations/V<XXX>__<description>.sql`.
2. Убедитесь, что версия `<XXX>` идет строго после последней существующей.
3. Добавьте SQL команды (CREATE, ALTER и т.д.).
4. Запустите `init_db.sh` для применения.

6.3 Фоновые задачи (Jobs)

Задачи хранятся в таблице `jobs` и обрабатываются сервисом `worker`.

6.3.1 Типы задач

- `report_generate`: Генерация PDF.
- `data_export`: Экспорт в CSV/Excel.
- `notification_send`: Отправка email.

6.4 Наблюдаемость (Observability)

- **Метрики**: Prometheus эндпоинт `/metrics` (в процессе).
- **Логирование**: Настроено через stdout/stderr, рекомендуется собирать через Vector/Loki.
- **Аудит**: Все чувствительные операции записываются в `audit_log`.

6.5 CI/CD

Проект использует **GitHub Actions** (`.github/workflows/`): - `ci.yml`: Сборка и прохождение всех тестов при каждом Push/PR. - `opencode.yml`: Проверки стиля и линтинг.

Последнее обновление: 2026-05-14